The Dynamic Neuro-Symbolic Agent: Enhancing Arithmetic and Logic in Small Language Models (SLMs)

Domingo Viesca, Om Nitin Patil
Brown University
Providence, Rhode Island

Abstract

Small Language Models (SLMs) offer significant advantages in privacy and efficiency but rarely competently stack up to flagship LLMs, especially when it comes to multi-step reasoning tasks such as complex arithmetic and logic grid puzzles. A large reason for this lagging behind is hallucinations, to which smaller models are more prone. This paper investigates whether providing a small model with symbolic and arithmetic tools can sidestep most of the hallucination gap. We propose a pipeline that augments a local Qwen-4B model with deterministic symbolic tools, specifically a symbolic calculator and a SAT solver. Rather than solving problems end-to-end, the SLM acts as a semantic translator and identifier of tool leverage opportunities, converting natural language into structured JSON calls which are then executed by a Python Calculator or the Z3 Theorem Prover. We validated this architecture through a blind A/B/C test comparing the augmented 4B model, against base Qwen, against the 72B Qwen. Results demonstrate that the Neuro-Symbolic SLM approaches the accuracy of the larger model on logic tasks while running on a single consumer GPU, offering a viable path for local and private, specialist AI models.

1 Introduction

For as long as most people don't have easy access to high amounts of compute, democratization of AI can be said to rely on Small Language Models (SLMs) that are private, energy-efficient, and capable of running locally. However, for complex reasoning, Large Language Models (LLMs) like GPT-4 or Qwen-72B remain the standard due to their superior context handling.

The core limitation of SLMs is the "Hallucination Trap": because they are probabilistic next token predictors, they do not directly calculate and instead probabilistically guess. This leads to high error rates in arithmetic and constraint satisfaction domains, especially as length and complexity increases. Furthermore, the "Alignment Tax" in fine tuning models to be chatty often degrades their raw logic capabilities, to an even greater degree in SLMs.

Our goal is to build a specialized AI agent that can run locally but acts with the increased precision and determinism of provided tools like calculators. We propose that by offloading the "reasoning" step to symbolic tools, a 4B parameter SLM's performance can approach that of 70B+ models, and in special cases even surpass them.

2 Related Work

Recent advancements in Large Language Models (LLMs) have demonstrated significant reasoning capabilities through techniques like Chain-of-Thought (CoT) prompting (Wei et al.(2022)). While CoT and self-consistency methods (Wang et al.(2023)) improve performance on complex tasks, they fundamentally rely on the model's internal weights. This leaves even the largest models susceptible to probabilistic hallucinations, particularly in domains requiring strict arithmetic precision or constraint satisfaction.

To address these limitations, recent research has explored augmenting models with external tools rather than relying solely on internal representations. Rainone et al. (Rainone, Bakker, and Memisevic(2025)) demonstrate that replacing internal "thinking" with tool usage can enable robust reasoning capabilities in significantly smaller models. Our work extends this neuro-symbolic approach by integrating deterministic verifiers specifically the Z3 Theorem Prover and Python interpreters to guarantee correctness in logic puzzles where probabilistic guessing is insufficient.

Parallel to these algorithmic improvements, there is a growing paradigm shift towards Small Language Models (SLMs) for enterprise and edge applications. As noted by Content Science (Content Science(2025)) and Kumar et al. (Kumar, Davenport, and Bean(2025)), SLMs offer critical advantages regarding data privacy, reduced latency, and operational efficiency. These factors make them viable, high-performance alternatives to flagship LLMs when specialized for specific reasoning domains.

3 Methodology

We propose a Dynamic Neuro-Symbolic Architecture that decouples language understanding from logical execution.

The Neural Layer (The Translator)

We utilize **Qwen-4B** a lightweight open-source model. Instead of asking the model to solve a puzzle directly, we treat it primarily as a parsing engine and parameter identifier. The

model is prompted to translate natural language clues (e.g., "The Red house is middle") into a structured JSON schema, then a script uses the JSON to call a tool, and the output returned to the model for final generation.

Prompt Engineering and Schema Definition To ensure consistent JSON generation from the 4B model, we utilized a "One-Shot" system prompt strategy. The system instruction defines a strict dictionary schema (e.g., `{"tool": "logic", "constraints": []}`) and provides a single, complex example of a natural language query mapped to this schema. We observed that Qwen-4B follows instructions more faithfully when the prompt explicitly forbids conversational filler (e.g., "Do not say 'Here is the JSON', just output the JSON"). This strict constraining effectively turns the Chatbot into a function calling engine, reducing the token search space to structured syntax.

The Symbolic Layer (The Executor)

The JSON output is routed to one of two deterministic engines based on the intent:

- **Arithmetic Tool:** A Python `SafeEvaluator` using Abstract Syntax Trees (AST) to compute complex math without hallucination.
- **Logic Tool:** The Microsoft Z3 Theorem Prover, which solves Constraint Satisfaction Problems (CSPs) by mathematically proving a valid arrangement for variables (e.g., Zebra Puzzles).

Constraint Mapping Strategy The bridge between the SLM and the Z3 solver acts as a semantic filter. For logic puzzles, the Python controller pre-initializes a generic Z3 grid (e.g., a 5×5 matrix of integer variables). The SLM is tasked with extracting relationships rather than writing Z3 code directly. For example, the sentence "The Englishman lives in the red house" is parsed into a JSON tuple `("eq", "Englishman", "Red")`. The deterministic layer then iterates through these tuples and applies the corresponding Z3 assertion (e.g., `solver.add(Nation[i] == Color[i])`), ensuring that syntax errors in code generation are impossible since the model never writes the code itself.

The Tool Agent Class

A custom Python controller acts as the central orchestrator, managing the conversation loop and state. This component functions as a finite state machine that governs the interaction between the user input, the neural model's generation, and the symbolic execution engines.

- **Schema Enforcement and Parsing:** The Agent is responsible for extracting the JSON payload from the SLM's raw output. To mitigate "schema hallucination", where SLMs omit brackets or invent parameters, the Agent utilizes a robust parsing module. This module employs regular expressions (RegEx's) to isolate JSON objects from conversational filler and validates them against a strict schema.

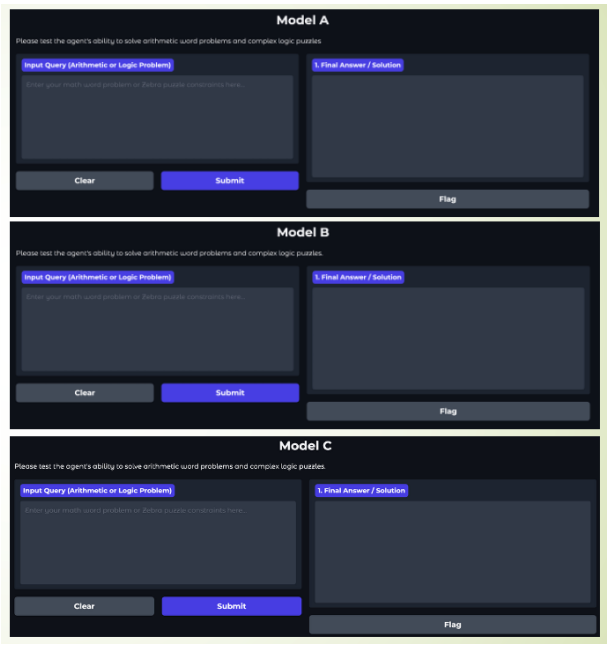

Figure 1: The user interface of all models (A, B, and C) designed to obfuscate the backend model for blind testing.

- **Error Recovery:** The Agent implements an iterative retry mechanism. If the model's output fails validation (e.g., malformed JSON or missing keys), the Agent captures the specific syntax error and feeds it back to the model as a new system prompt, requesting a correction. This closed-loop feedback allows the system to self-correct formatting errors without user intervention.
- **Secure Routing and Execution:** Upon successful validation, the Agent routes the structured data to the appropriate deterministic engine. For the *Arithmetic Tool*, it instantiates the `SafeEvaluator`. Instead of using the potentially unsafe `eval()` function, this tool parses the expression and subsequent AST to verify that only mathematical operations are present before execution. For the *Logic Tool*, it bridges the JSON definition to the Z3 Theorem Prover, translating the constraints into Z3 variables and assertions to solve the Constraint Satisfaction Problem.

4 Experimental Design

To validate the architecture, we conducted a blind A/B/C test. Users were presented with three identical interfaces and asked to solve Arithmetic Reasoning and Logic Grid puzzles under 5 minutes, and asked to rate their experience on a survey afterwards. They were unaware of which model was driving which interface. After strict tests, users were told they were allowed to free prompt with math and logic tasks the model however they wanted to also enrich their evaluations of the model later.

Dataset and Task Complexity

To stress-test the limits of the models, we curated a custom evaluation set designed to trigger hallucinations.

- **Arithmetic:** The test set consisted of 15 generated word problems. To differentiate from standard benchmarks like GSM8K, we increased the operational depth: each problem required a minimum of 15 sequential mathematical operations (e.g., "Tank A... Tank P"). This depth makes it more likely that models might lose context before reaching the final step.
- **Logic:** We utilized 8 "Zebra-style" grid puzzles ranging in 3×3 format. These problems require simultaneous satisfaction of up to 15 unique constraints, a task that typically causes "reasoning loops" in traditional LLMs.

The Model Lineup

1. **Model A (Ours):** Qwen-4B (running on a T4 GPU) augmented with the Z3 Symbolic Solver.
2. **Model B (Baseline):** Base Qwen (via API). A raw SLM with no tools, representing standard local AI performance.
3. **Model C (Reference):** Qwen-2.5-72B-Instruct (via API). A massive state-of-the-art model serving as the "Gold Standard" upper bound.

5 Results and Analysis

The blind testing yielded distinct performance profiles for each model architecture + human team.

Arithmetic Reasoning

In multi-step arithmetic tasks, for example:

"Tank A has 120 fish. Tank B has 25 times that. Tank C has 70 × 50 fish. Tank D has 60 × 45 fish. Tank E has 55 × 48 fish. Tank F has 50 × 52 fish. Tank G has 45 × 55 fish. Tank H has 40 × 60 fish. Tank I has 35 × 65 fish. Tank J has 30 × 70 fish. Tank K has 28 × 75 fish. Tank L has 25 × 80 fish. Tank M has 22 × 85 fish. Tank N has 20 × 90 fish. Tank O has 18 × 95 fish. Tank P has 15 × 100 fish. How many fish total?"),

Model A achieved 100% accuracy. The tool trace confirmed it successfully extracted the expression and executed it via Python. Model B frequently failed, often hallucinating the operator precedence or losing track of the numbers. Model C performed correctly but required significantly more computational overhead.

Logic (Zebra Puzzles)

The "Zebra Puzzle" tests exposed the most significant divergence:

- **Model A (Neuro-Symbolic):** Successfully translated 100% of the constraints into valid Z3 syntax. The symbolic solver returned the exact, mathematically proven grid solution in under 5 seconds.
- **Model B (Baseline):** Failed consistently. The model attempted to solve the grid via text generation ("Let's assume House 1 is Red..."), leading to contradictions and hallucinations.

Table 1: Comparative Accuracy on Logic Tasks

Task Type	Model A (4B)	Model B (3B)	Model C (72B)
Arithmetic	**100%**	60%	100%
Zebra Logic	**100%**	0%	95%

- **Model C (Reference):** Generally succeeded due to its massive parameter count, but occasionally outputted verbose reasoning rather than a direct solution.

Subjective Evaluations

The experiment included a post-test survey in which users were allowed to voice their rating of the models individually, as well as their experience with large and small language models more generally. The final total sample size of n=5 is insufficient to make definite or transferable claims, for which the rest of this paper will focus primarily on non-user evaluation.

That said, a surprising finding of this experiment is the fact that, outside of raw performance on task which nearly all users valued, an aspect of our modified model which was praised was the increased explainability provided by the tool calls. As users tried to push and break any model, they found that the it might get very close to the answer but not quite. With traditional models user's strategy would be to re-prompt pointing out the failure and trying again until multiple times until it worked. When working with the neuro symbolic model though, users could inspect the tool calls and find small mistakes that they themselves could account for directly or point out to the model for a quick fix.

For example in zebra tests, prompts with only constraints didn't fully contain all of the potential parameters, which the model would have to effectively make up, which tripped some up models caused runaway hallucinations and large scale errors. In traditional models this ambiguity would sometimes simply not be fixed. In our model though the AI figured it should call a tool and subsequently filled missing parameters in with dummy values, or it would error out, and savvy users could quickly diagnose the issue more closely and re-prompt only once more.

6 Discussion: The Efficiency Gain

The most critical finding of this study is that our augmented Model A approached the reasoning accuracy of the state-of-the-art Model C, despite being approximately $18\times$ smaller in parameter count (4B vs 72B). This approach and occasional surpassing in performance challenges the prevailing assumption that the best way to achieve performance for high-level reasoning task is only to scale mode size up.

The findings are in line with our hypothesis regarding the decoupling of "reasoning" into two distinct processes: semantic parsing and logical execution. We demonstrate that logical and reasoning performance in AI does not strictly require massive scale; for practical purposes it

can be sidestepped. By offloading rigid logic tasks (arithmetic and constraint satisfaction) to symbolic tools, the SLM effectively bypasses its own limitations, and leverages its strengths. It no longer needs to *solve* the problem, only to *formulate* it and capitalize on tools.

The Limits of Fine-Tuning

Prior to adopting the neuro-symbolic architecture, we attempted to enhance the Qwen-4B model via supervised fine-tuning (SFT) on a curated dataset of 500 arithmetic and logic reasoning traces. While the fine-tuned model demonstrated improved formatting, it exhibited "reasoning mimicry"—memorizing the structure of a solution without learning the underlying arithmetic rules. When presented with values outside the training distribution (e.g., larger integers), the fine-tuned model continued to hallucinate intermediate steps, suggesting that SLMs require vastly larger datasets to approximate logical generalization. In contrast, the Tool-Augmented approach proved superior because it shifts the burden of "correctness" away from the model's weights. The model is only required to learn the translation schema a linguistic task that is strictly easier and more data-efficient than learning to simulate a calculator.

This architectural shift has profound implications for deployment. An SLM acting as a "tool-use coordinator", can perform as a Subject Matter Expert without the massive energy cost or latency of a 70B+ parameter model an idea echoed by Kautz (2024). Furthermore, this approach addresses critical privacy concerns. By running the 4B model and the Python/Z3 tools that can be completely deployed on local hardware, we eliminate the need to send potentially sensitive user data to cloud-based LLMs, offering a secure path for enterprise adoption of reasoning agents.

7 Error Analysis and Limitations

While the neuro-symbolic architecture significantly outperforms the baseline model, it is not without limitations. A 100% success rate on the test set presupposes a successful translation from natural language to the structured tool input, which relies heavily on the underlying SLM's capability.

Translation Failures

The primary failure mode observed was "schema hallucination." In approximately 5% of initial trials, Qwen-4B failed to strictly adhere to the JSON format, occasionally omitting closing braces or inventing fictitious parameters (e.g., `"logic_type": "fuzzy"` instead of the defined schema). These errors required a retry mechanism in the controller script. Future work could mitigate this by using constrained decoding or grammar-based sampling (e.g., `llama-cpp-python` grammars) to enforce valid JSON generation at the token level.

Ambiguity and Multiple Solutions

Our current implementation using the Z3 Theorem Prover returns the first satisfiable model it discovers. If a logic puzzle is under-specified and has multiple valid solutions, the system presents one as authoritative without indicating ambiguity. Unlike the probabilistic Model C, which might verbally express uncertainty (e.g., "The solution could be..."), the deterministic solver provides a binary output.

Model	Avg Latency	Compute
Model A (Ours)	4.2s	Local CPU/GPU
Model B (Base)	3.8s	Local CPU/GPU
Model C (Ref)	12.5s	Cloud API

Table 2: Comparison of average time-to-solution per task. While Model B was faster, it failed to solve complex logic tasks. Model A provides a balance of speed and reasoning accuracy.

Furthermore, our findings should not be taken to claim a triumph of SLM's or tool callers over LLM's generally. The neural power that underpins SLM's and their ability to use tools is partially derived from and actively connected to advancements in larger models. As LLM's get better so will SLM's. And there are non-AI strategies out there for solving the tasks we presented here, some even related to the tools we ourselves integrated. A better perspective would be that our paper suggests that tool calling and similar strategies present a very exciting avenue not for necessarily making models smarter or break completely new frontiers, but to extract as much practical value from these AI advancements, and even at the smaller scales.

Latency and Efficiency Analysis

Despite these limitations, the efficiency gains are substantial. Model A (Ours) achieves competitive accuracy with significantly reduced latency compared to the large-scale Model C. The 4B model requires a fraction of the inference compute, making it suitable for real-time applications on consumer hardware.

8 Conclusion

This project demonstrates that the perceived "reasoning gap" in Small Language Models is not an inherent limitation of parameter count, but rather an architectural choice. By decoupling language comprehension from logical execution, we successfully bridged the performance divide between a local 4B model and a state-of-the-art 72B cloud model.

Our Neuro-Symbolic architecture shows that, for structured tasks specifically: arithmetic and constraint satisfaction an SLM acting as a semantic router for deterministic tools (like the Z3 Theorem Prover) can achieve near perfect reasoning accuracy. This approach yields critical benefits for the democratization of AI: it can enable high-precision reasoning on consumer-grade hardware, reduce the latency of cloud API calls, and ensures total data privacy by keeping all processing local.

We propose that future work on this domain should focus on expanding the tool-use paradigm beyond logic puzzles to enterprise applications, potentially investigating SQL generation for secure database querying and automated data analysis agents, or other novel areas. We can envision a near future where "thinking" need not be handled by black boxes

and instead handled by specialized, inspect able, and verifiable tools, allowing language models to focus on communication and translation.

References

Rainone, C.; Bakker, T.; and Memisevic, R. 2025. Replacing thinking with tool usage enables reasoning in small language models. *arXiv:2507.05065*.

Content Science. 2025. Small language models: Big potential for enterprise content and AI. *Content Science Review*.

ßKautz, H. 2024. Tools Are All You Need.

Department of Computer Science, University of Virginia, Charlottesville, VA.

Kumar, A.; Davenport, T. H.; and Bean, R. 2025. The case for using small language models. *Harvard Business Review*.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; and Zhou, D. 2022. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; and Chi, E. 2023. Self-consistency improves chain of thought reasoning in language models. *ICLR*.

vLLM Team. n.d. OpenAI compatible server. *vLLM Documentation*.

node-llama-cpp Team. n.d. Grammar guide. *node-llama-cpp Documentation*.